

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

Members:

YADAV Anshuman Krishna (exercise 2, 4)

MAHALINGAM Nithees (exercise 1)

SARAVANAN Arun Prasath (exercise 3)

Github Link:

https://github.com/anshuman-krishna/advanced-algorithms-and-programming/tree/LAB_02_Revision

Exercise 1: MAHALINGAM Nithees

Friend Request Timeline

Input: A text message

Output:

- Number of uppercase letters
- Number of urgency punctuation marks (!, ?)
- Capital letter ratio
- Final message classification

Begin

Initialize uppercase_letters to 0

Initialize punctuation_marks to 0

Initialize total_letters to 0

Set previous_character to null

Set consecutive_count to 1

For each character in the message:

If the character is a letter:

Increase total_letters by 1

If the letter is uppercase:

Increase uppercase_letters by 1

If the character is '!' or '?':

Increase punctuation_marks by 1

If the character is the same as previous_character:

Increase consecutive_count by 1

Else:

Reset consecutive_count to 1

Update previous_character to the current character

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

```
If total_letters equals 0:
Set caps_ratio to 0
Else:
Set caps_ratio to uppercase_letters divided by total_letters
If caps_ratio is at least 0.6 OR punctuation_marks is at least 5:
classification = "AGGRESSIVE"
Else if caps_ratio is at least 0.3 OR punctuation_marks is at least 3:
classification = "URGENT"
Else:
classification = "CALM"

Return all computed values
End
```

Complexity Analysis Questions:

1. How many character comparisons are needed for an n-character string?

Answer:

The algorithm checks the message once. Therefore, for each character, we perform a constant number of checks:

- Check if character is alphabetic -> hence, 1 comparison
- If alphabetic, check if uppercase -> 1 comparison
- Check if character is '!' or '?' -> will take up to 2 comparisons
- Check if character equals previous character -> it takes 1 comparison
- Possibly check if repeat count > 3 -> takes 1 comparison

So for each character, approx 5-7 comparisons are made.

Therefore, total comparisons : **c.n**

2. Can this be done in a single pass? What is the minimal number of passes required?

Yes.

All of the required information:

- Uppercase count
- Alphabetic count

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

- Punctuation count
- Repeated character detection

is collected simultaneously during one traversal of the string.

Therefore, Minimal number of passes required : **1**

3. How would the complexity change if we support Unicode beyond ASCII?

Answer:

Time Complexity : **O(n)** (still)

Because:

- We still process each character once.
 - Character classification (isalpha(), isupper()) remains constant-time per character.
-
-

Exercise 2: YADAV Anshuman Krishna

Mutual Friends Detection Using Sets

A. Intersection

```
FUNCTION Intersection(set1, set2) -> Set
result <- empty set
FOR EACH element IN set1 DO
IF set2.Contains(element) THEN
result.Add(element)
END IF
END FOR
RETURN result
```

B. Difference

```
FUNCTION Difference(set1, set2) -> Set
result <- empty set
FOR EACH element IN set1 DO
IF NOT set2.Contains(element) THEN
result.Add(element)
END IF
END FOR
RETURN result
```

C. Union

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

```
FUNCTION Union(set1, set2) -> Set
result <- empty set
FOR EACH element IN set1 DO
result.Add(element)
END FOR
FOR EACH element IN set2 DO
IF NOT result.Contains(element) THEN
result.Add(element)
END IF
END FOR
RETURN result
```

D. Jaccard

```
FUNCTION JaccardSimilarity(set1, set2) -> Real
inter <- Intersection(set1, set2)
uni <- Union(set1, set2)

inter_size <- (inter)
uni_size <- (uni)
IF uni_size = 0 THEN
RETURN 0.0
END IF
RETURN inter_size / uni_size
```

E. Friend Suggestion

```
FUNCTION SuggestFriends(user_friends, all_users_friends) -> Set
suggestions <- empty set
FOR EACH friend_id IN user_friends DO
friends_of_friend <- all_users_friends[friend_id]

FOR EACH candidate IN friends_of_friend DO
IF NOT user_friends.Contains(candidate) AND candidate != current_user_id THEN
suggestions.Add(candidate)
END IF
END FOR
END FOR
RETURN suggestions
```

Complexity Analysis Questions:

1. What is the time complexity of set intersection for sets of size m and n?

Answer:

As we learnt in Lecture 4, the time complexity of set intersection for sets of size m and n would be:

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

- $O(\min(m,n))$ on average
- $O(m * n)$ for worst case scenario, and
- $O(1)$ for best case scenario

2. How does the representation (sorter array vs. hash set vs. bit array) affect complexity?

Answer:

Hash Set: Has fast lookups ($O(1)$) and Intersection is $O(m+n)$ or $O(\min(m,n))$. Hence, making it good for sparse data.

Sorted Array: Lookups take $O(\log n)$ and Intersection takes $O(m+n)$ because we iterate through both arrays at the same time.

Bit Array: Has extremely fast bitwise operations (AND, OR). Intersection is $O(U/W)$, where U is the total number of possible users and W is word_size. This generally is constant time relative to the number of friends but depends on the size.

3. For a network with millions of users, which representation would you choose and why?

Answer:

I would choose **Hash Set**, because, as mentioned above, it is good for sparse data, in a real social network the matrix is sparse, even with millions of users a normal person generally has friends ranging between 200 to 600.

If we compare it with other representations for the same case:

- Bit Array: It would require a bit array of size million for every user, that will require higher computational values.
- Sorted Array: They are generally good for the scenario but inserting a new friend would be slow, around $O(N)$ due to shifting elements.
- Hash Set: Only stores the amount of friends, and adding a new friend is instant.

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

Exercise 3: SARAVANAN Arun Prasath

FriendRecommendation by Common Interests

Input: Users[U][I], FriendList, userID, K

Output: Top K similar users and recommended interests

```
BEGIN
CREATE empty list scores
FOR u <- 0 TO U-1 DO
  IF u != userID AND u NOT IN FriendList[userID] THEN
    dot <- 0
    normA <- 0
    normB <- 0

    FOR i <- 0 TO I-1 DO
      dot <- dot + Users[userID][i] * Users[u][i]
      normA <- normA + Users[userID][i]^2
      normB <- normB + Users[u][i]^2
    END FOR

    IF normA != 0 AND normB != 0 THEN
      sim <- dot / (rootnormA * rootnormB)
    ELSE
      sim <- 0
    END IF
    ADD (u, sim) TO scores
  END IF
END FOR
SORT scores in descending order
TopK <- first K users from scores
CREATE rec[I] <- 0

FOR EACH (u, sim) IN TopK DO
  FOR i <- 0 TO I-1 DO
    IF Users[userID][i] = 0 THEN
      rec[i] <- rec[i] + Users[u][i] * sim
    END IF
  END FOR
END FOR
RETURN TopK, rec
END
```

Complexity Analysis Questions:

1. What is the time complexity of computing all pairwise similarities for U users and I interests?

Answer:

To compute pairwise similarities:

- We compare every user with every other user -> this gives $U \times U = U^2$ comparisons.
- For each comparison, cosine similarity requires scanning all I interests -> $O(I)$.

Therefore, the total time complexity is $O(U^2 \times I)$

2. How would you optimize this for sparse matrices (most users have few interests)?

Answer:

If most values in the matrix are 0:

- Instead of storing the full matrix, store only non-zero interests using a hash map or sparse representation.
- Compute the dot product only on common interests.
- Use an inverted index (interest -> list of users) to avoid unnecessary comparisons.

This reduces computation significantly. Optimized time complexity becomes approximately $O(E)$. Where E is the number of non-zero entries.

3. What is the space complexity of storing the full matrix vs. sparse representations?

Answer:

- Full matrix stores all values -> $O(U * I)$
- Sparse representation stores only non-zero values -> $O(E)$

Where E is the number of non-zeros entities .

II.2415 Advanced Algorithms and Programming

Lab 2: Revision of Basic Data Structures: Team T24

Exercise 4: YADAV Anshuman Krishna

Mutual Followers Matrix

A. Structure

```
STRUCTURE FollowerMatrix
matrix <- 2D Array of Boolean
size <- Integer
END STRUCTURE
```

```
FUNCTION InitializeGraph(n) -> FollowerMatrix
graph <- new FollowerMatrix
graph.size <- n
graph.matrix <- Create 2D Boolean Array of size n x n
RETURN graph
```

B. Follow and Unfollow

```
PROCEDURE Follow(graph, follower, followee)
graph.matrix[follower][followee] <- TRUE
END PROCEDURE
```

```
PROCEDURE Unfollow(graph, follower, followee)
graph.matrix[follower][followee] <- FALSE
END PROCEDURE
```

```
FUNCTION Is_following(graph, follower, followee) -> Boolean
RETURN graph.matrix[follower][followee]
```

C. Following Unfollowing List

```
FUNCTION Get_followers(graph, user) -> Array
followers_list <- empty list
n <- graph.size
```

...

to be continued in T24_LAB2_BFM